Phase 1 — Setup & Planning: Detailed Actionable Guide

> ***Goal:*** *Get fully organized — environment, repo, graph schema, anchor companies, data strategy, default labeling, and task split — before writing a single line of project code. This phase produces the blueprint that every subsequent phase builds on.*

Why Phase 1 Matters

Traditional credit-risk evaluation treats firms as **isolated entities** and completely misses the systemic risk embedded in production and trade networks. This project exists to fix that.

Phase 1 is where you make every foundational decision — what graph structure to use, which companies to start with, where to get data, and how to label defaults. If any of these are wrong, every later phase inherits that error. Think of Phase 1 as **drawing the architectural blueprint before laying a single brick**.

The deliverables from Phase 1 (graph schema, anchor company list, data source plan, folder structure) become the **direct inputs** to Phase 2, where you actually collect and clean data. Without a finalized graph schema, Phase 2 has no target structure to populate. Without a curated anchor list, you don't know which SEC filings to parse or which yfinance tickers to query.

---

1. Environment & Repository Setup

1.1 Install & Verify Tools

Ensure the following are installed on your machine. Open a terminal and verify each:

Tool	Install From	Verify Command
Python 3.10+	python.org or Anaconda	`python --version` → must show 3.10+
Git	git-scm.com	`git --version`
VS Code	code.visualstudio.com	Open it, install Python + Jupyter extensions
Node.js 18+	nodejs.org (LTS)	`node --version`

> *[!TIP] You already have a conda environment set up (from a previous conversation). Activate it with* `conda activate scncc` *before proceeding.*

Why these tools?

- **Python 3.10+** is the standard for ML/data work and is required by PyTorch Geometric (Phase 4).
- **Git** enables version control and collaboration — non-negotiable for any multi-person project.
- **VS Code** with the Python + Jupyter extensions gives you an integrated editor, debugger, and notebook runner.
- **Node.js 18+** is needed for the React frontend dashboard (Phase 6), but installing it now avoids setup friction later.

1.2 Create the Shared GitHub Repository

1. Go to github.com/new

2. **Repository name:** `supply-chain-credit-contagion`
3. **Visibility:** Private (you can make it public later)
4. Check "Add a README file"
5. Add `.gitignore` → select **Python**
6. License → **MIT** (standard for academic projects)
7. Click **Create repository**
8. Clone it locally:

```
git clone https://github.com/<your-username>/supply-chain-credit-contagion.git
cd supply-chain-credit-contagion
```

9. Add your teammate as a collaborator: **Settings → Collaborators → Add people**

Why MIT license? It's the most permissive open-source license — anyone can use, modify, and distribute your work. Perfect for academic projects that you might later publish or share with professors.

1.3 Create the Folder Structure

Your repo should have this exact structure. Create every folder now (even if empty — add a `.gitkeep` file in empty folders so Git tracks them):

```
supply-chain-credit-contagion/
├── data/
│   ├── raw/                ← unprocessed CSVs, SEC filings, Kaggle downloads
│   ├── processed/          ← cleaned, normalized, merged datasets
│   └── external/           ← WIOD tables, Crunchbase data, macro indicators
├── graph/
│   ├── build_graph.py      ← (create later) graph construction script
│   ├── entity_resolution.py
│   └── network_metrics.py
├── model/
│   ├── baseline/           ← logistic regression, random forest
│   ├── gnn/                ← GCN, GAT, GraphSAGE, PI-GNN code
│   └── evaluation/         ← metrics, confusion matrices, ROC plots
├── simulation/
│   ├── siis_model.py       ← SIIS epidemic contagion
│   ├── shock.py            ← SCR-NR / SCR-ER protocols
│   └── stress_test.py
├── api/                    ← FastAPI backend
├── frontend/               ← React dashboard
├── notebooks/              ← Jupyter experiments & EDA
├── tests/                  ← unit tests & integration tests
├── docs/                   ← project documentation
├── environment.yml         ← conda environment spec
└── README.md
```

Why this structure?

- **`data/raw` vs `data/processed`:** Keeps untouched source data separate from cleaned versions. You never modify raw data — this makes your pipeline reproducible. If something goes wrong, you can always re-process from raw.
- **`graph/`:** Separates graph-building logic from ML modeling. Graph construction (Phase 3) is a distinct engineering step.

- **`model/baseline/` vs `model/gnn/`:** You always build a simple baseline first (logistic regression, Phase 4A) before the complex GNN (Phase 4B). Keeping them separate enforces this discipline.
- **`simulation/`:** The SIIS contagion engine (Phase 5) is a standalone simulation system — it takes the trained model's outputs and runs "what-if" scenarios.
- **`api/` and `frontend/`:** Backend (FastAPI) and frontend (React) are separate concerns. The API serves data; the frontend visualizes it.
- **`notebooks/`:** Jupyter notebooks are for exploration only. Production code goes in the appropriate module folder.
- **`tests/`:** Automated tests ensure code reliability as the project scales.

To create all folders at once (run from repo root in PowerShell):

```
$folders = @(
    "data/raw", "data/processed", "data/external",
    "graph", "model/baseline", "model/gnn", "model/evaluation",
    "simulation", "api", "frontend", "notebooks", "tests", "docs"
)
foreach ($f in $folders) {
    New-Item -ItemType Directory -Path $f -Force
    if (-not (Test-Path "$f/.gitkeep")) { New-Item -ItemType File -Path "$f/.gitkeep" -Force
}
}
```

Then commit:

```
git add -A
git commit -m "chore: initialize project folder structure"
git push origin main
```

1.4 Set Up Branch Strategy

Agree on a simple branching model for two people:

Branch	Purpose
`main`	Always stable, working code
`dev`	Integration branch — merge features here first
`feature/<name>`	Individual task branches (e.g., `feature/data-collection`, `feature/api-setup`)

Why this model?

- You never push broken code to `main`. The `dev` branch is your testing ground.
- Feature branches let two people work on different parts (data collection and API setup) simultaneously without stepping on each other's code.
- When a feature is complete and tested on `dev`, it gets merged to `main`.

Create the `dev` branch now:

```
git checkout -b dev
git push origin dev
```

---

2. Define the Graph Schema — `G = (V, E, X, W)`

This is the **most important technical decision** of Phase 1. Your entire project builds on this schema. Every phase — data collection, graph building, GNN training, contagion simulation — references this definition.

2.1 What Is a Graph Schema?

Your supply chain is modeled as a **heterogeneous, directed, weighted, multi-relational property graph**. In mathematical notation: `G = (V, E, X, W)`.

Here's what each component means and why it exists:

Component	Mathematical Definition	Domain	Description
V (Nodes)	Set of `n` corporate entities	`	V
E (Edges)	Set of directed edges `(v_i, v_j, r)`	`	E
X (Features)	Node feature matrix `X ∈ ℝ^{n × d}`	`d ≈ 16`	Financial metrics + topological features per node
W (Weights)	Edge weight matrix `W_ij ∈ [0, 1]`	Normalized	Relationship intensity (trade volume, dependency ratio)
A (Adjacency)	`A ∈ {0,1}^{n × n}` with `A_ij = 1` if `(v_i, v_j) ∈ E`	Binary	Captures graph topology for GNN message passing
D (Degree)	Diagonal matrix `D_ii = Σ_j A_ij`	Integer	Node degree for Laplacian normalization
L (Laplacian)	`L = D - A` (unnormalized), `L_sym = D^{-1/2} L D^{-1/2}`	Real	Spectral basis for graph convolutions

Why "heterogeneous"? Because your graph has multiple node types (OEMs, Tier 1 suppliers, raw material providers) and multiple edge types (supply, trade credit, ownership). A simple homogeneous graph can't represent this richness.

Why "directed"? Because supply relationships have a direction. Denso *supplies* Toyota — not the other way around. The direction determines how risk propagates: if Denso fails, Toyota loses a supplier (upstream disruption).

Why "weighted"? Because not all relationships are equally important. Toyota sourcing 60% of its chips from TSMC is far more critical than sourcing 2% of its rubber seals from a small vendor. Edge weights `W_ij` encode this asymmetric dependency.

2.2 Nodes `V` — Corporate Entities

Each node represents **one company**. Every node gets a **UUID** (assigned during entity resolution in Phase 2).

Node Attribute	Type	Description	Source
`entity_id`	`UUID`	Unique identifier across all data sources	Generated
`name`	`string`	Canonical company name	Entity resolution
`ticker`	`string`	Stock ticker (if publicly traded)	yfinance

`sector`	`string`	Industry sector (e.g., "Automotive OEM")	Manual / SIC code
`tier`	`enum`	`OEM`, `Tier1`, `Tier2`, `Tier3`, `RawMaterial`	Manual classification
`country`	`string`	HQ country (ISO 3166-1 alpha-2)	yfinance / manual
`is_public`	`bool`	Whether the company is publicly traded	Manual

Why UUIDs? The same company appears differently across data sources (e.g., "Toyota Motor Corp", "Toyota Motor Corporation", "TOYOTA", "TM"). A UUID ensures a single canonical identity regardless of name variations. Entity resolution (Phase 2) maps all variations to one UUID.

Why `tier`? The automotive supply chain has a clear tiered structure: OEMs (Toyota, Ford) buy from Tier 1 suppliers (Denso, Bosch), who buy from Tier 2 (specialty steel mills, chip makers), and so on. This tier classification helps you analyze how risk cascades down the supply chain levels.

2.3 Node Features $X \in \mathbb{R}^{|V| \times d}$ — Financial + Topological

These are the **features your GNN will actually learn from**. They split into two categories:

Intrinsic Features (from financial data — 10 dimensions):

These capture each company's *own* financial health — their **idiosyncratic vulnerability** before any network contagion effects:

#	Feature	Formula / Source	Why It Matters
1	`revenue`	Total annual revenue	Company scale — larger firms may absorb shocks better
2	`total_debt`	Total debt outstanding	Absolute leverage exposure
3	`market_cap`	Share price × shares outstanding	Market's collective valuation of the firm
4	`current_ratio`	Current Assets / Current Liabilities	Short-term liquidity — can it pay bills due within a year?
5	`debt_to_equity`	Total Debt / Shareholder Equity	Leverage ratio — how much the firm relies on borrowed money
6	`profit_margin`	Net Income / Revenue	Profitability — is the firm making money or burning it?
7	`revenue_growth`	YoY % change in revenue	Growth trajectory — growing firms are healthier
8	`interest_coverage`	EBIT / Interest Expense	Can it service its debt? Below 1.5× is dangerous
9	`altman_z_score`	Composite bankruptcy predictor (see Section 5)	A single-number default risk proxy
10	`roa`	EBIT / Total Assets	Asset efficiency — how well the firm uses what it owns

Extrinsic Features (from network topology — 6 dimensions):

These are computed *from the graph itself* in Phase 3 — they capture each company's **structural importance** in the supply chain:

#	Feature	NetworkX Function	Why It Matters
11	`in_degree`	`G.in_degree(node)`	How many suppliers feed this company — more suppliers = redundancy
12	`out_degree`	`G.out_degree(node)`	How many customers depend on it — more customers = systemic importance
13	`betweenness_centrality`	`nx.betweenness_centrality(G)`	Is this company a bridge/bottleneck? High value = removing it disconnects the network
14	`eigenvector_centrality`	`nx.eigenvector_centrality(G)`	A node is important if its neighbors are also important (transitive influence)
15	`pagerank`	`nx.pagerank(G)`	Random-walk importance — like Google's algorithm applied to supply chains
16	`clustering_coefficient`	`nx.clustering(G.to_undirected())`	Local redundancy — high clustering means many alternative supply paths exist

> ***Total feature dimension:*** `d = 16` ***per node*** *(you can add more later, but start with these)*

Why both intrinsic and extrinsic? A company might be financially healthy (great intrinsic features) but structurally critical (high betweenness centrality) — its failure would cascade widely. Conversely, a financially weak firm might be structurally peripheral (low degree, low centrality) — its failure affects few others. The GNN needs both perspectives to make accurate predictions.

2.4 Edges `E` — Relationship Types

Your graph is a **heterogeneous multi-relational directed graph**. Three edge types:

Edge Type	Direction	Meaning	Risk Propagation	Example
`supply`	Supplier → Customer	A supplies parts/materials to B	Upstream failure → downstream disruption	Denso → Toyota
`trade_credit`	Creditor → Debtor	A extends payment terms to B	Default → accounts receivable write-off	Bosch → BMW (net-60 terms)
`ownership`	Parent → Subsidiary	A owns equity in B	Contagion via equity exposure	Toyota → Denso (24.7% stake)

> *[!IMPORTANT]* ***Edge direction matters.*** *For* `supply` *edges, the arrow goes* ***Supplier → Customer*** *(A supplies B). This means risk propagates in the* ***reverse*** *direction — if Supplier A fails, Customer B suffers. Your GNN's message passing will naturally handle this bidirectional information flow.*

Why three edge types? Credit contagion doesn't travel through a single channel:

1. **Supply disruption** — if your only chip supplier goes bankrupt, your production stops.

2. **Trade credit default** — if a customer owes you $50M under net-60 terms and files for bankruptcy, you just lost $50M in accounts receivable.
3. **Equity contagion** — if Toyota owns 24.7% of Denso and Denso's value drops, Toyota's balance sheet takes a direct hit.

Each edge type transmits risk differently, at different speeds, and with different magnitudes. A heterogeneous graph lets the GNN learn these distinct transmission patterns.

2.5 Edge Weights `W` — Relationship Intensity

Edge Attribute	Type	Description	How to Estimate
`weight`	`float [0,1]`	Normalized relationship intensity	See formula below
`relationship_type`	`enum`	`supply`, `trade_credit`, `ownership`	Manual / NLP extraction
`data_source`	`string`	Where this edge came from	`sec_10k`, `kaggle`, `manual`
`confidence`	`float [0,1]`	How sure you are this edge exists	1.0 for verified, 0.7 for NLP-extracted
`last_verified`	`date`	When this relationship was last confirmed	Filing date

Weight formula (for supply edges):

```
w_ij = purchase_volume_ij / total_COGS_j
```

This means: "*What fraction of Company B's total cost of goods sold (COGS) comes from Supplier A?*" If A provides 40% of B's inputs, `w_ij = 0.4` — a very strong dependency.

If you don't have exact volumes (likely for most edges), start with `weight = 1.0` for all edges and refine later. The GNN can still learn relative importance from the graph topology.

Why track `confidence`? Not all edges are equally reliable. An edge verified from an SEC 10-K filing (`confidence = 1.0`) is much more trustworthy than one extracted by NLP from a news article (`confidence = 0.7`). This metadata helps you filter or weight edges during model training.

2.6 Schema Summary Diagram

```
[Company A] ──supply(w=0.3)──→ [Company B]
   │                              │
   ├── entity_id: uuid            ├── entity_id: uuid
   ├── name: "Denso Corp"         ├── name: "Toyota Motor"
   ├── ticker: "6902.T"           ├── ticker: "TM"
   ├── tier: Tier1                ├── tier: OEM
   ├── X = [rev, debt, ...]       ├── X = [rev, debt, ...]
   └── default_label: 0           └── default_label: 0
```

---

3. Choose Anchor Companies (The Starting 30)

3.1 Why Automotive Manufacturing?

You need to pick a sector where supply chain data is publicly available, relationships are well-documented, and there's academic precedent for this type of analysis:

Factor	Automotive	Electronics	Pharma
SEC filing availability	⭐ ⭐ ⭐ Excellent	⭐ ⭐ ⭐ Excellent	⭐ ⭐ Good
Clear tiered structure	⭐ ⭐ ⭐ Very clear	⭐ ⭐ Moderate	⭐ Weak
Public supply chain data	⭐ ⭐ ⭐ Best in class	⭐ ⭐ Good	⭐ Limited
Historical disruptions	⭐ ⭐ ⭐ (COVID, chips, Suez)	⭐ ⭐	⭐ ⭐
Academic precedent	⭐ ⭐ ⭐ Most studied	⭐ ⭐	⭐

Verdict: Automotive is the best choice. It has the clearest tiered structure (OEM → Tier 1 → Tier 2 → Raw Materials), the most publicly available supply chain disclosures, and multiple recent disruptions (2011 Fukushima earthquake, 2021 chip shortage, 2024 Red Sea route disruption) that you can validate your model against. Stick with it.

3.2 Your 30 Anchor Companies

> *[!NOTE] These are your **seed nodes**. In Phase 2, you will expand to ~300 by exploring 10-K filings of these anchors and discovering their suppliers/customers. Each 10-K filing typically names 5–15 suppliers, so 30 anchors can generate 150–450 total edges.*

OEMs (Original Equipment Manufacturers) — 10 companies

These are the final assemblers — the companies whose logos appear on the cars:

#	Company	Ticker	Country	Why Include
1	Toyota Motor	`TM`	Japan	World's largest automaker by volume; extensive public disclosures
2	Volkswagen Group	`VWAGY`	Germany	Europe's largest; owns Audi, Porsche, Lamborghini, etc.
3	General Motors	`GM`	USA	Major US OEM; heavy domestic supply chain; detailed SEC filings
4	Ford Motor	`F`	USA	Major US OEM; excellent 10-K disclosure quality
5	Stellantis	`STLA`	Netherlands	Owns Chrysler, Fiat, Peugeot, Jeep — broad cross-continental supply chain
6	Honda Motor	`HMC`	Japan	Major global OEM with well-documented supply networks
7	BMW Group	`BMWYY`	Germany	Premium segment; high vertical integration
8	Hyundai Motor	`HYMTF`	South Korea	Fast-growing; aggressive EV transition reveals new dependency patterns
9	Tesla	`TSLA`	USA	EV leader; uniquely vertically integrated supply chain

10	Nissan Motor	NSANY	Japan	Alliance with Renault; well-documented supply chain disruption history

Tier 1 Suppliers — 12 companies

These supply assembled systems directly to OEMs (braking systems, wiring harnesses, electronics modules):

#	Company	Ticker	Country	What They Supply
11	Robert Bosch*	Private	Germany	Automotive electronics, braking, powertrain — world's largest auto supplier
12	Denso Corp	DNZOY	Japan	Thermal, powertrain, electronics; 24.7% Toyota-owned (ownership edge!)
13	ZF Friedrichshafen*	Private	Germany	Transmissions, chassis, safety systems
14	Magna International	MGA	Canada	Contract manufacturing, mirrors, closures — supplies nearly every OEM
15	Continental AG	CTTAY	Germany	Tires, braking, electronics, ADAS
16	Aisin Corp	ASEKY	Japan	Drivetrain, body, electronics — Toyota Group member
17	Lear Corporation	LEA	USA	Seating, e-systems
18	Aptiv	APTV	Ireland (HQ)	Vehicle wiring, connectors, ADAS — critical electronics supplier
19	BorgWarner	BWA	USA	Turbochargers, drivetrain, electrification components
20	Yazaki*	Private	Japan	Wire harnesses, instrumentation — supplies most global OEMs
21	Hyundai Mobis	HYUMY (OTC)	South Korea	Modules, chassis, infotainment
22	Faurecia / Forvia	FURCF (OTC)	France	Interiors, seating, clean mobility

> **Companies marked with * are **private** — no ticker. You will need to manually enter financial data from their published annual reports and press releases. This is expected — many critical Tier 1 suppliers are private companies.*

Tier 2 / Raw Materials / Adjacent — 8 companies

These supply components, raw materials, or services to Tier 1 suppliers:

#	Company	Ticker	Country	Role in Supply Chain
23	CATL	300750.SZ	China	Largest EV battery manufacturer globally
24	Panasonic Holdings	PCRFY	Japan	Tesla's primary battery partner; electronics

25	Nucor Steel	`NUE`	USA	Major automotive-grade steel supplier
26	Alcoa Corp	`AA`	USA	Aluminum for vehicle lightweighting
27	TSMC	`TSM`	Taiwan	Automotive chip fabrication — the 2021 shortage epicenter
28	NXP Semiconductors	`NXPI`	Netherlands	Automotive-specific chips (radar, security, in-vehicle networking)
29	Flex Ltd	`FLEX`	Singapore	Contract electronics manufacturing
30	BASF	`BASFY`	Germany	Automotive coatings, catalysts, engineering plastics

3.3 How to Record This

Create a spreadsheet or CSV called `data/raw/anchor_companies.csv`:

```
entity_id,name,ticker,country,tier,sector,is_public
,Toyota Motor,TM,JP,OEM,Automotive,TRUE
,Volkswagen Group,VWAGY,DE,OEM,Automotive,TRUE
,General Motors,GM,US,OEM,Automotive,TRUE
...
```

Leave `entity_id` blank for now — you'll assign UUIDs during entity resolution in Phase 2. The CSV becomes the input to Phase 2's financial data pull (yfinance queries each ticker) and entity resolution pipeline.

---

4. Evaluate & Plan Your Data Sources

Before collecting any data, you need to know **exactly** what each source provides, what it costs, and how difficult it is to use. This prevents wasted effort in Phase 2.

4.1 Data Source Evaluation Matrix

Source	What It Gives You	Cost	Difficulty	Priority
SEC EDGAR 10-K	Supplier/customer names from filings (Item 1, 1A, 7)	Free	Medium (NLP needed for scale)	🔴 Critical
yfinance	Financial metrics (revenue, debt, ratios) for public companies	Free	Easy (Python API)	🔴 Critical
Company annual reports	Financial data for private companies (Bosch, ZF, Yazaki)	Free	Manual extraction	🔴 Critical
Kaggle datasets	Pre-built supply chain networks (search "supply chain")	Free	Easy	🟡 Supplementary

FactSet Revere	Curated supplier-customer relationships	Paid (trial available)	Easy	🟡 Nice-to-have
WIOD	Macro-level inter-sector trade flows for shock parameters	Free	Medium	🟢 Later (Phase 5)
Panjiva / ImportGenius	Customs/shipment records → infer transaction volumes	Paid	Hard	🟢 Stretch goal
CSMAR	Auto supplier lists (Chinese market coverage)	Institutional	Medium	🟢 Supplementary

Why prioritize these? The 🔴 Critical sources (SEC EDGAR, yfinance, annual reports) are all **free** and provide exactly the two things you need: (1) edges (who supplies whom — from SEC filings) and (2) node features (financial metrics — from yfinance/annual reports). You can build a complete, functional graph using only these three sources. Everything else is supplementary.

4.2 SEC EDGAR — How to Actually Use It

This is your **primary** source for building edges (supply relationships). Here's the exact process:

1. **Go to:** efts.sec.gov/LATEST/search-index
   - This is the full-text search endpoint for EDGAR filings
2. **Search for each anchor company** by name or CIK number
3. **Open their latest 10-K filing** (annual report)
4. **Ctrl+F** search within the filing for these keywords:
   - `"major supplier"`, `"significant supplier"`, `"key supplier"`
   - `"principal customer"`, `"major customer"`, `"significant customer"`
   - `"supply agreement"`, `"supply chain"`, `"sole source"`
   - `"purchase commitment"`, `"raw material"`
5. **For each supplier/customer named**, add a row to your edge list CSV

Why these specific keywords? SEC regulations require companies to disclose "major" or "significant" suppliers/customers (generally those representing >10% of revenue or costs). The keywords above are the exact phrases companies use in their filings. Items 1 (Business Description), 1A (Risk Factors), and 7 (MD&A) are where these disclosures typically appear.

> *[!TIP]* ***Start manual, automate later.*** *For Phase 1, just manually extract suppliers from 5–10 of your anchor companies' 10-K filings. In Phase 2, you'll write NLP scripts (using spaCy NER) to scale this across hundreds of filings.*

4.3 yfinance — Quick Financial Data Check

To verify tickers work, open a Python shell and test:

```
import yfinance as yf
stock = yf.Ticker("TM")  # Toyota
print(stock.info.get("shortName"))      # "Toyota Motor Corporation"
print(stock.info.get("totalRevenue"))    # should be ~$280B
print(stock.info.get("debtToEquity"))    # leverage
print(stock.info.get("currentRatio"))    # liquidity
```

If a ticker returns `None` for key fields, you may need an alternate ticker (e.g., some Japanese companies use Tokyo Stock Exchange tickers like `7203.T` instead of ADR tickers like `TM`).

Why test now? If tickers are broken or API rate limits hit you, you want to discover this in Phase 1 — not mid-way through Phase 2 when you're trying to pull data for 300 companies. Early testing surfaces data quality issues cheaply.

---

5. Default Labeling Strategy

Your GNN needs a **target variable** — which companies have defaulted or are at high risk? This is the `default_label` column in your data.

5.1 The Problem

Actual corporate defaults are **rare events** (<2% per year). With ~300 companies, you may have fewer than 10 real defaults in your dataset. That's not enough for a model to learn from.

5.2 The Solution: Multi-Strategy Labeling

Use **three complementary approaches** to label your `default_label` column. Each addresses a different aspect of default risk:

Strategy 1: Altman Z-Score (Primary — compute from yfinance data)

The Altman Z-score is a well-established bankruptcy predictor developed by NYU professor Edward Altman in 1968. It has been validated across decades and industries. You can compute it entirely from data available via yfinance:

$$Z = 1.2 \cdot X_1 + 1.4 \cdot X_2 + 3.3 \cdot X_3 + 0.6 \cdot X_4 + 1.0 \cdot X_5$$

Where:

- X_1 = Working Capital / Total Assets — measures **liquidity** (can the firm cover short-term obligations?)
- X_2 = Retained Earnings / Total Assets — measures **cumulative profitability** (has the firm historically generated profits?)
- X_3 = EBIT / Total Assets — measures **operating efficiency** (how productively does the firm use its assets?)
- X_4 = Market Cap / Total Liabilities — measures **solvency** (does the market value exceed what it owes?)
- X_5 = Revenue / Total Assets — measures **asset turnover** (how efficiently does the firm generate sales from its assets?)

Labeling thresholds:

Z-Score Range	Zone	Default Label	Interpretation
Z > 2.99	Safe Zone	`0` (healthy)	Low bankruptcy risk
1.81 < Z ≤ 2.99	Grey Zone	`0` (healthy, but flagged)	Uncertain — monitor closely
Z ≤ 1.81	Distress Zone	`1` (default proxy)	High bankruptcy risk

Why use Z-Score as primary? It's computable entirely from free data (yfinance), doesn't require paid credit rating services, and has a well-established empirical track record. It gives you enough `default_label = 1` cases (distressed companies) to train a model, even though actual bankruptcies are rare.

Strategy 2: Credit Rating Downgrade Proxy

If a company's credit rating has been downgraded to **BB+ or below** (non-investment grade / "junk"), label it as `1`. This supplements the Z-Score with market-expert assessments.

Sources:

- S&P ratings (search `"<company name> credit rating"`)
- Moody's (search `"<company name> Moody's rating"`)
- Free lookup at tradingeconomics.com/bonds

Why BB+ as the threshold? BB+ is the boundary between "investment grade" (BBB- and above) and "speculative grade" (BB+ and below). This is where institutional investors are often forced to sell bonds, triggering a liquidity crisis that can push the firm toward default.

Strategy 3: Market-Implied Distress (Merton Distance-to-Default)

Use **Merton Distance-to-Default (DtD)** as a continuous risk score. This treats a company's equity as a call option on its assets — when asset value drops below debt obligations, the firm defaults:

- DtD < 1.5 → high distress → `default_label = 1`
- DtD > 3.0 → healthy → `default_label = 0`

Why include this? The Z-Score uses accounting data (often 3–6 months stale). The Merton model uses market data (stock price, volatility), which reflects information in near-real-time. Combining both gives you labels that capture both accounting fundamentals and market sentiment.

> *[!IMPORTANT]* ***For Phase 1, just use Strategy 1 (Altman Z-Score).*** *It's computable from free data and gives you enough labels to start. You'll add Strategies 2 and 3 in Phase 4 when you have more data and need more sophisticated labels.*

---

6. Task Split & Weekly Sync Setup

6.1 Role-Based Task Assignment for Phase 2

Once Phase 1 is complete, divide Phase 2 work by skillset:

Phase 2 Task	Who	Deliverable
Extract supplier names from SEC 10-K filings (manual first, then NLP)	ML Person	`data/raw/supply_chain.csv`
Pull financial data via yfinance for all 30 anchors	ML Person	`data/raw/company_financials.csv`
Compute Altman Z-Scores and assign default labels	ML Person	`default_label` column in financials CSV
Set up SQLite database schema	Developer	`data/scncc.db` with tables
Write CSV → SQLite loader script	Developer	`api/load_data.py`
Set up FastAPI with health-check endpoint	Developer	`api/main.py` with `/` endpoint
Research and list additional companies from SEC filings	Both	Expand from 30 → 100+ companies

Why this split? The ML person focuses on data quality and feature engineering — the inputs to the model. The developer focuses on infrastructure — the systems that store, serve, and display that data. Both roles investigate SEC

filings, but from different angles (ML person extracts supply relationships; developer designs the database schema to store them).

6.2 Weekly Sync Meeting Template

Set a recurring **30-minute weekly call**. Use this agenda:

```
SCNCC Weekly Sync — [Date]
──────────────────────────

1. Progress Check (10 min)
   - ML Person: What did you finish? What's blocked?
   - Developer: What did you finish? What's blocked?

2. Data Review (10 min)
   - How many companies/edges do we have now?
   - Any entity resolution issues?
   - Any data sources that aren't working?

3. Next Week's Goals (5 min)
   - ML Person's top 3 tasks for next week
   - Developer's top 3 tasks for next week

4. Technical Decisions (5 min)
   - Any schema changes needed?
   - Any tool/library decisions to make?
```

Why weekly? Less than weekly and issues fester. More than weekly and you lose productive coding time. 30 minutes forces you to be concise and focus on blockers — the things that actually slow the project down.

How Phase 1 Connects to Phase 2

Everything you produced in Phase 1 flows directly into Phase 2:

Phase 1 Output	→ Phase 2 Input
Graph schema `G = (V, E, X, W)`	Defines the target structure for data collection — you know exactly what tables/columns to create
30 anchor companies with tickers	Seed list for yfinance financial data pulls and SEC 10-K filing searches
Data source evaluation	Prioritized action plan — which APIs to call first, which filings to read
Default labeling strategy (Altman Z-Score)	Algorithm for computing the `default_label` target variable from financial data
Folder structure & repo	All data outputs from Phase 2 go into `data/raw/` and `data/processed/`
Task split	Each person knows exactly what to build in Phase 2

Phase 2 will populate the empty graph schema with real data — extracting supplier–customer relationships from SEC filings, pulling financial metrics via APIs, and resolving entity names into a unified index. The outputs of Phase 2 (two

key CSVs: `supply_chain.csv` and `company_financials.csv`, plus a master entity index) become the raw ingredients for graph construction in Phase 3.

Phase 1 Completion Checklist

By the end of Phase 1, **all** of the following should be done:

Environment & Tools:

- [ ] Python 3.10+, Git, VS Code, Node.js installed and verified
- [ ] Conda environment `scncc` activated and working
- [ ] environment.yml or `requirements.txt` updated if new packages are needed

Repository & Collaboration:

- [ ] GitHub repository created with correct folder structure (all folders from Section 1.3)
- [ ] Branching strategy agreed on (`main` → `dev` → `feature/*`)
- [ ] Teammate added as collaborator
- [ ] Initial folder structure committed and pushed

Graph Schema:

- [ ] Graph schema `G = (V, E, X, W)` fully defined (as documented in Section 2)
- [ ] All 16 node features (10 intrinsic + 6 extrinsic) listed and understood
- [ ] All 3 edge types (supply, trade_credit, ownership) defined with directionality
- [ ] Edge weight formula decided (`w_ij = purchase_volume_ij / total_COGS_j`)

Anchor Companies:

- [ ] 30 anchor companies identified with tickers
- [ ] Anchor list saved in `data/raw/anchor_companies.csv`
- [ ] Automotive sector selected with justification documented in `docs/decisions.md`

Data Sources:

- [ ] Data source evaluation completed (know what you'll use for edges vs. features)
- [ ] Quick yfinance test — verified tickers work for at least 10 companies
- [ ] SEC EDGAR — read at least 2–3 10-K filings manually, found supplier names
- [ ] Private company data plan — identified annual reports for Bosch, ZF, Yazaki

Default Labeling:

- [ ] Default labeling strategy decided (Altman Z-Score as primary)
- [ ] Z-Score formula understood and documented
- [ ] Labeling thresholds set (Z ≤ 1.81 → `default_label = 1`)

Project Management:

- [ ] Task split for Phase 2 written down (who does what)
- [ ] Weekly sync meeting scheduled (calendar invite sent)
- [ ] All decisions committed to `docs/decisions.md` in the repo

> *[!CAUTION]* ***Don't skip the manual SEC filing review.*** *Reading even 2–3 real 10-K filings yourself gives you intuition for what the NLP pipeline needs to extract in Phase 2. You'll understand the format, the language, and the challenges (ambiguous company names, indirect references, footnotes) — this cannot be replaced by reading about it.*